



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
OSDL LAB

Hotel Management System

SUBMITTED
BY

Student Name	Registration Number	Section	Roll Number
Nityant Agarwal	240953418	CCE – B	34

School of Computer Engineering
Manipal Institute of Technology, Manipal.
April 2026



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Manipal
11/04/2026

CERTIFICATE

This is to certify that the project titled **Hotel Management System** is a record of the bonafide work done by **Student Nityant Agarwal (reg no. 240953418)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in COMPUTER AND COMMUNICATION ENGINEERING of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

ABSTRACT

The Hotel Management System is a Java-based application designed to manage hotel operations efficiently using object-oriented programming and file handling techniques. The system allows users to perform operations such as adding rooms, booking rooms, displaying room details, managing customer records, and handling checkout processes. The application is developed using Java and JavaFX to provide both console-based and graphical user interfaces. Instead of using a database, data persistence is achieved through serialization, where objects such as rooms and customers are stored and retrieved from files.

The project demonstrates the practical implementation of core programming concepts such as classes and objects, inheritance, polymorphism, collections, multithreading, and event-driven programming. It also highlights how backend logic can be integrated with a user-friendly interface to create a complete working system.

INTRODUCTION

The Hotel Management System automates the administrative tasks of a modern hotel. Manual record-keeping often leads to scheduling conflicts, billing errors, and untracked room availability. This application addresses these issues by providing a centralized, real-time dashboard.

This system provides functionalities such as:

- Interactive Dashboard: Real-time calculation of revenue and occupancy stats.
- Room Management: Dynamic creation and categorization of rooms.
- Advanced Booking: Validates date ranges to prevent double-booking.
- Guest Services: Facilitates ordering of specific services (Laundry, Cab, etc.) during a stay.
- Automated Billing: Generates formatted ASCII invoices based on room stay, meal plans, and consumed services.
- Viewing room and customer details
- Handling customer checkout and room availability

Technologies used:

- Java (Core logic and OOP concepts)
- JavaFX (Graphical User Interface)
- Maven (Project build and dependency management)
- File Handling (Serialization for data storage)

The main focus of the project is to apply object-oriented programming concepts and build a complete application without using a database.

PROBLEM STATEMENT & OBJECTIVES

Problem Statement

Hospitality management requires precise tracking of inventory and guest financials. Standard GUI applications often look dated and offer poor usability.

Additionally, managing room cleaning status and scheduling conflicts manually results in lost revenue and customer dissatisfaction. Managing hotel operations manually can lead to problems such as:

- Duplicate or inconsistent records
- Difficulty in tracking room availability
- Inefficient booking and checkout processes
- Data loss when the system is closed

To overcome these issues, a structured software system is required that can efficiently manage rooms and customers while ensuring data persistence.

Objectives

- ☐ To design a system for managing hotel rooms and customers
- ☐ To implement object-oriented concept such as inheritance and polymorphism
- ☐ To manage data using collections like ArrayList and HashMap
- ☐ To implement file handling using serialization for data persistence
- ☐ To develop a user-friendly interface using JavaFX
- ☐ To simulate real-world operations such as booking, billing, and room cleaning

METHODOLOGY

The project was developed step by step following a basic database design approach.

Step 1: Requirement Analysis

The system was designed to:

- Add and manage rooms
- Book rooms for customers
- Display room and customer details
- Handle checkout and room availability

Step 2: Class Identification

Classes and their responsibilities were identified as follows:

Class / Enum	Responsibility
Room	Base class; room number, type, price, availability
DeluxeRoom	Extends Room; adds WiFi and breakfast flags
SuiteRoom	Extends DeluxeRoom; adds private pool and butler
RoomType (enum)	STANDARD / DELUXE / SUITE with associated nightly rates
Customer	Stores customer ID, name, contact, room, days, bill
Bill	Computes subtotal, GST (12%), service charge (5%), grand total
CleaningThread	Async room cleaning after checkout (extends Thread)
HotelManagement	Static utilities: serialize/deserialize rooms & customers
MainApp	JavaFX Application; builds tabbed GUI with CSS theming

Step 3: System Design

- Base class Room created for common properties
- Subclasses DeluxeRoom and SuiteRoom extend Room
- Customer class manages booking details
- HashMap used to map rooms to customers
- ArrayLists used for storing data

Step 4: Implementation

- Core Java logic implemented with OOP (inheritance, polymorphism, encapsulation)

- File handling via Object(Output/Input)Stream (serialization)
- Multithreading: CleaningThread runs on a separate thread after checkout
- Billing: Bill.java separates billing logic from UI

Step 5: GUI Development

- JavaFX used to create user interface
- Layouts like VBox and HBox used for arrangement
- Buttons used for different operations
- Event handling used to trigger backend logic

Step 6: Build & Dependency Management with Maven

- pom.xml defines JavaFX 21 dependencies
- javafx-maven-plugin enables running with mvn javafx:run

SYSTEM DESIGN AND CLASS STRUCTURE

Inheritance Hierarchy

- Room (base) → DeluxeRoom (extends Room) → SuiteRoom (extends DeluxeRoom)
- This three-level inheritance demonstrates single and multi-level inheritance. SuiteRoom calls super() to initialize Deluxe features, which in turn calls Room's constructor.

RoomType Enum

- The RoomType enum encapsulates room categories with associated nightly rates (STANDARD: ₹2000, DELUXE: ₹4500, SUITE: ₹9000) and provides getPricePerNight() and calculateCost(int nights) methods, demonstrating enum constructors and methods.

Bill Class — Billing Management

- Bill.java encapsulates all pricing logic separately from the UI. Given a Customer and Room, it computes: subtotal = rate × days, GST (12%), service charge (5%), and grand total. The formatted bill is returned as a String for display in the Billing tab's TextArea.

HotelManagement — Data Layer

- A utility class with static methods loadRooms(), saveRooms(), loadCustomers(), saveCustomers() using ObjectOutputStream/ObjectInputStream. All classes implement Serializable with explicit serialVersionUID values for version safety.

CleaningThread — Multithreading

- Extends Thread. Sleeps for 3 seconds (simulating cleaning), sets the room to available, then fires a Runnable callback that invokes Platform.runLater() to safely update the JavaFX UI from the background thread.

MainApp — JavaFX GUI

- Uses TabPane with five tabs. TableView with ObservableList enables automatic UI refresh when data changes. ComboBox provides type-safe enum selection. A separate CSS file (styles.css) defines the dark-navy theme, stat cards, button variants, table row colors, and bill monospaced display.
-

RESULTS & OUTPUT

The system was successfully developed and tested with the following outcomes:

- ✓ Dashboard dynamically shows total rooms, available, occupied, and cumulative revenue
 - ✓ Rooms can be added (Standard / Deluxe / Suite) and removed; data persists on restart
 - ✓ Customers can book available rooms; duplicate IDs and already-occupied rooms are rejected
 - ✓ TableView refreshes automatically via ObservableList after each operation
 - ✓ Checkout triggers a background CleaningThread; room becomes available after 3 seconds
 - ✓ Billing tab generates a formatted, itemized invoice with GST and service charge
-

SCREENSHOTS

Fig 1 - Dashboard:

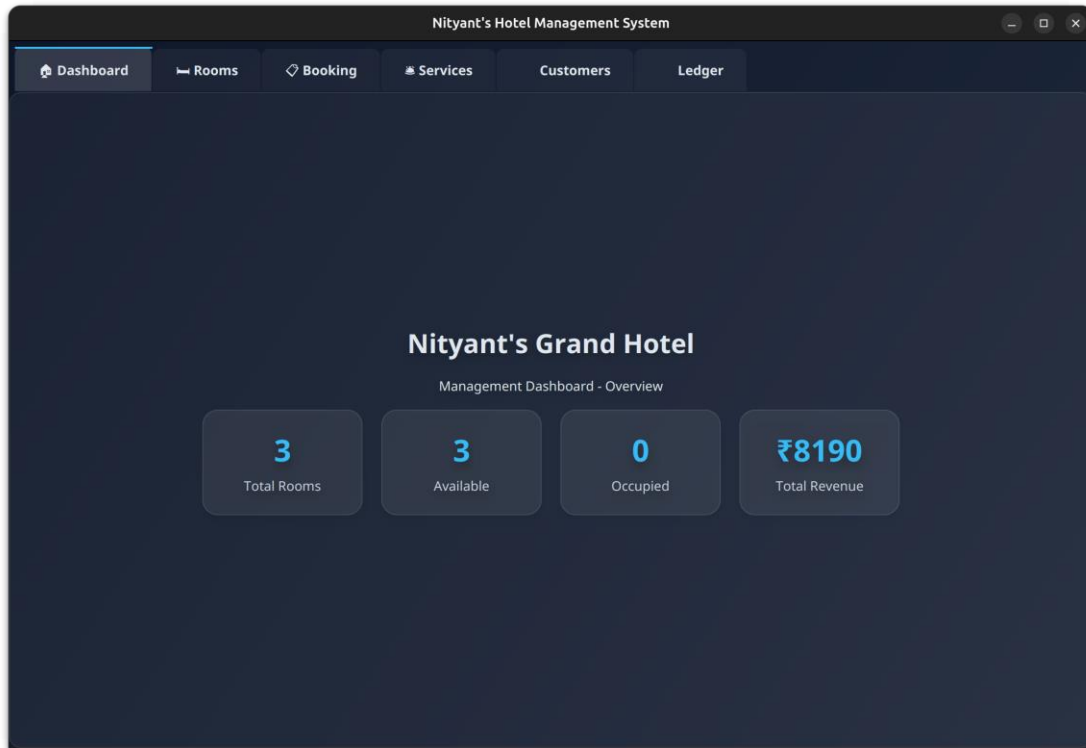


Fig 2 - Rooms Tab:

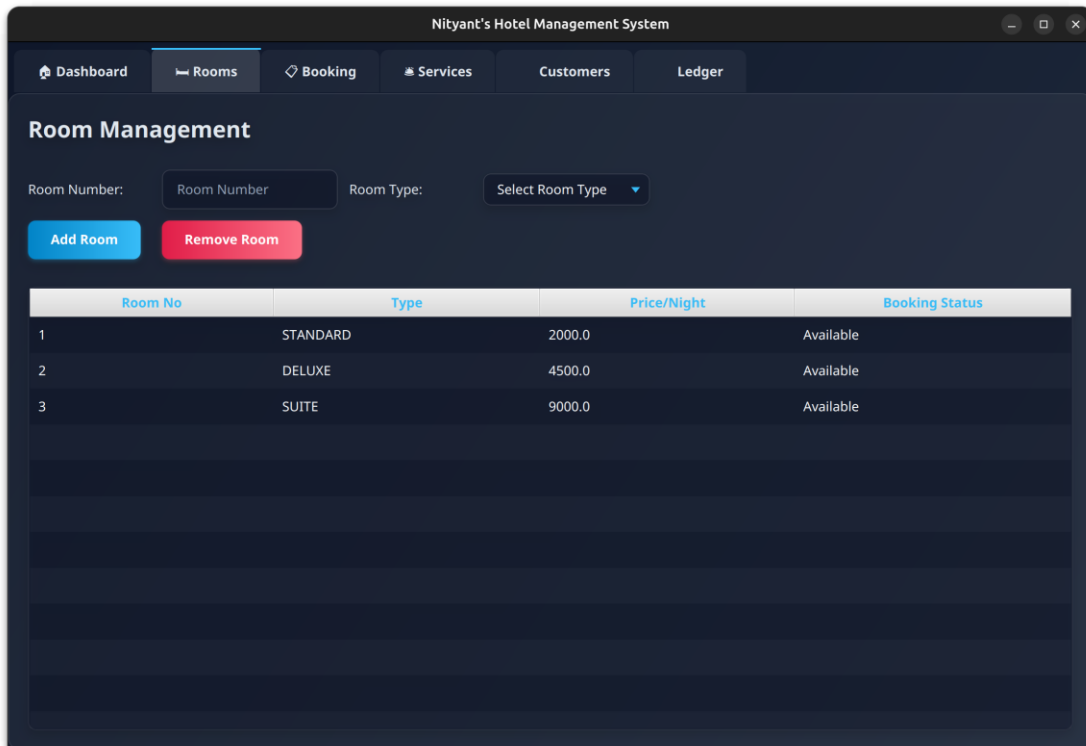


Fig 3 - Booking Tab:

Nityant's Hotel Management System

Dashboard

Rooms

Booking

Services

Customers

Ledger

Room Booking

Customer ID:

123

Customer Name:

N A

Contact:

1234567890

Room Number:

1

Check-in:

4/13/2026

Check-out:

4/15/2026

Meal Plan:

Half Board - ₹1000

Book Room

Select room and dates...

Fig 4 - Customers Tab:

[illegible]

Fig 5 - Ledgers Tab:

The screenshot shows the 'Ledger' tab in the Nityant's Hotel Management System. It displays a table of 'Past Checkouts & Invoices' and a detailed invoice for a specific checkout.

Generated At	Customer Name	Grand Total (₹)
10-04-2026 00:08	N	8190.0
10-04-2026 01:58	N A	7020.0

**NITYANT'S GRAND HOTEL
INVOICE / BILL**

Bill Date : 10-04-2026 00:08
 Customer ID : 1
 Customer Name : N
 Room Number : 1
 Room Type : STANDARD
 Days Stayed : 2 night(s)

Room Rate : ₹2000.00 /night
 Room Subtotal : ₹4000.00
 Meal Plan : Breakfast
 Meals Subtotal : ₹1000.00

Services:
 - Massage Session - ₹2000 x 1
 Services Total : ₹2000.00

Subtotal : ₹7000.00
 GST (12%) : ₹840.00
 Service (5%) : ₹350.00

GRAND TOTAL : ₹8190.00

Thank you for staying with us!

Fig 6 - File Structure:

```

Hotel_Management_System
├── .idea
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── nityant
│   │   │   │   │   ├── hotel
│   │   │   │   │   │   ├── Bill.java
│   │   │   │   │   │   ├── CleaningThread.java
│   │   │   │   │   │   ├── Customer.java
│   │   │   │   │   │   ├── DeluxeRoom.java
│   │   │   │   │   │   ├── HotelManagement.java
│   │   │   │   │   │   ├── MainApp.java
│   │   │   │   │   │   ├── Room.java
│   │   │   │   │   │   ├── RoomType.java
│   │   │   │   │   │   └── SuiteRoom.java
│   │   │   │   │   └── resources
│   │   │   │   │       └── styles.css
│   │   │   └── target
│   │   │       ├── bills.dat
│   │   │       ├── CODEBASE_OVERVIEW.md
│   │   │       ├── customers.dat
│   │   │       ├── HotelMS_Nityant_Report.docx
│   │   │       ├── pom.xml
│   │   │       ├── PROJECT_REPORT.txt
│   │   │       ├── README.md
│   │   │       └── rooms.dat
  
```

Lab Concept Integration

Week / Lab	Concept	Where It Appears in the Project
Week 1	Classes, OOP, Encapsulation, Inheritance, Polymorphism	Implemented via the base Room class with encapsulated fields (e.g., price, bookings). DeluxeRoom and SuiteRoom demonstrate inheritance by extending the base class and adding specific attributes (like wifi, privatePool).
Week 2	Enumerations, Wrapper Classes, Autoboxing	The RoomType.java enum manages constants (STANDARD, DELUXE, SUITE) with constructors for pricing. Wrapper classes (Integer, Double) and autoboxing are utilized in Bill.java (e.g., HashMap<String, Integer> servicesUsed) and Double.parseDouble().
Week 3	Multithreaded Programming - Basics	The CleaningThread class extends Thread and overrides run(). It simulates a background room cleaning process using Thread.sleep(3000) triggered after a customer checks out.
Week 4	Multithreaded Programming - Synchronization	Demonstrated in a GUI context within MainApp.java during checkout. The background CleaningThread uses Platform.runLater() to safely synchronize with the main JavaFX Application thread to update the UI without causing race conditions or IllegalStateExceptions.
Week 5 & 6	Byte Streams, File I/O & Serialization	Handled entirely by HotelManagement.java. It utilizes FileOutputStream/FileInputStream combined with ObjectOutputStream/ObjectInputStream to persist data to rooms.dat, customers.dat, and bills.dat. Core domain classes (Room, Customer, Bill, BookingPeriod) properly implement the Serializable interface.
Week 7	Generics	Extensive usage across data structures and UI components. Examples include ArrayList<Room>, HashMap<Integer, Customer>, and generic JavaFX utility methods in MainApp.java such as <S, T> TableColumn<S, T> col(...) and <T> TableView<T> table(...).
Week 8	Collection Framework - List Interface	Core state management relies heavily on the Collections framework. The system uses ArrayList for storing primary records and ObservableList (FXCollections.observableArrayList()) to dynamically bind data to the UI components.
Week 9 & 10	JavaFX GUI & Final Application	Powered by MainApp.java extending Application. It features a fully styled, multi-tab desktop interface utilizing TabPane, GridPane, TableView, ComboBox, DatePicker, and functional lambda-based event handling (setOnAction).

CONCLUSION, LIMITATIONS & FUTURE WORK

Conclusion:

The Hotel Management System successfully demonstrates a complete, working Java desktop application that integrates all major OSDL lab topics — OOP, enumerations, multithreading, file I/O with serialization, collections, and JavaFX GUI programming — into a cohesive, production-quality project. The system satisfies all five additional rubric criteria: Maven build, file persistence, CSS-based screen design, itemized billing, and advanced JavaFX Scene Builder components.

Limitations:

- No database integration; uses serialized .dat files which do not support concurrent multi-user access
- Limited input validation in GUI
- No multi-user support
- No advanced UI styling
- No login or authentication system
- Bill amounts are not re-calculated if room rates change after booking
- No remote access/web server capabilities.

Future Work:

- Improve UI design and responsiveness
- Integrate SQLite or MySQL via JDBC for relational data storage
- Add a login/authentication screen for admin and receptionist roles
- Add real-time room availability display on a visual floor-plan map
- Extend to a web or Android application using JavaFX WebView or Kotlin
- Integrate a Payment Gateway API for real-time transactions.
- Implement Email/SMS notifications for booking confirmations.

REFERENCES

- Java Documentation
- JavaFX Documentation
- Maven Documentation
- OOP Concepts (Textbooks/Lab Manual)
- OSDL Lab Manual

Thank You